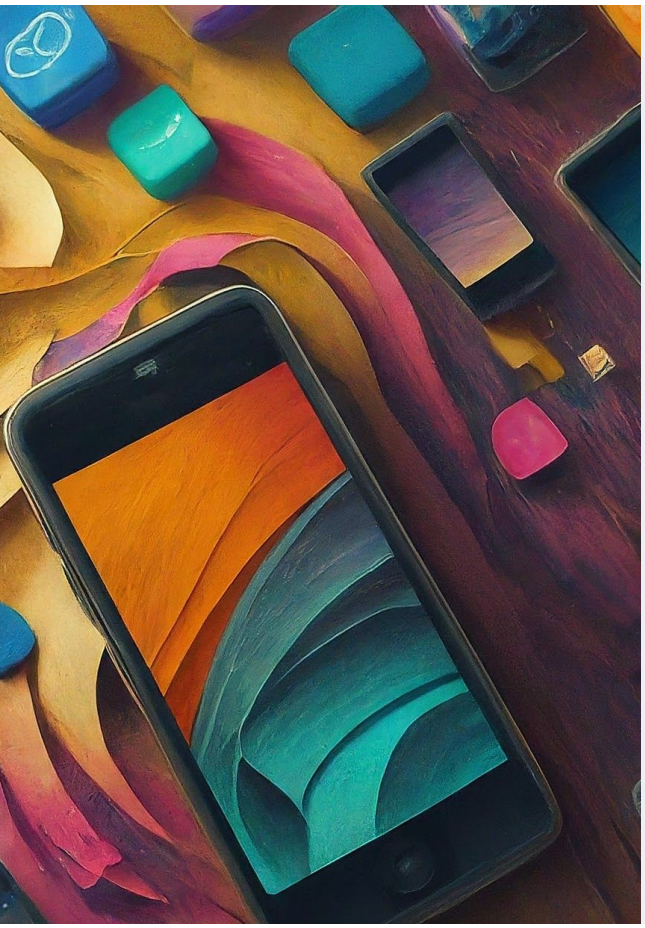


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS I

Ementa

Tipos de dispositivos móveis. Emuladores de dispositivos móveis. Sistemas operacionais móveis. Plataformas nativas. Introdução ao Material Design. Ambientes para desenvolvimento de aplicações móveis. Linguagens para desenvolvimento de aplicações móveis. Kits de desenvolvimento de software (SDKs) para aplicações móveis nativas. Persistência de Dados locais e remotos. Acesso a Geolocalização, câmera e outros recursos. Consumo e criação de APIs. Desenvolvimento Dirigido a testes (TDD). Controle de versionamento.

Competências Profissionais

- Avaliar as abordagens de desenvolvimento híbrido, multiplataforma ou nativo e selecionar a melhor abordagem para atender a necessidade do cliente.
- Utilizar linguagens de programação para dispositivos móveis e raciocínio lógico adequados para resolução de situações problema e ou desenvolvimento de projetos diversos.

Bibliografia Básica

- DEITEL, H.; DEITEL, A. Android para Programadores. 2 ed. Porto Alegre: Bookman, 2015.

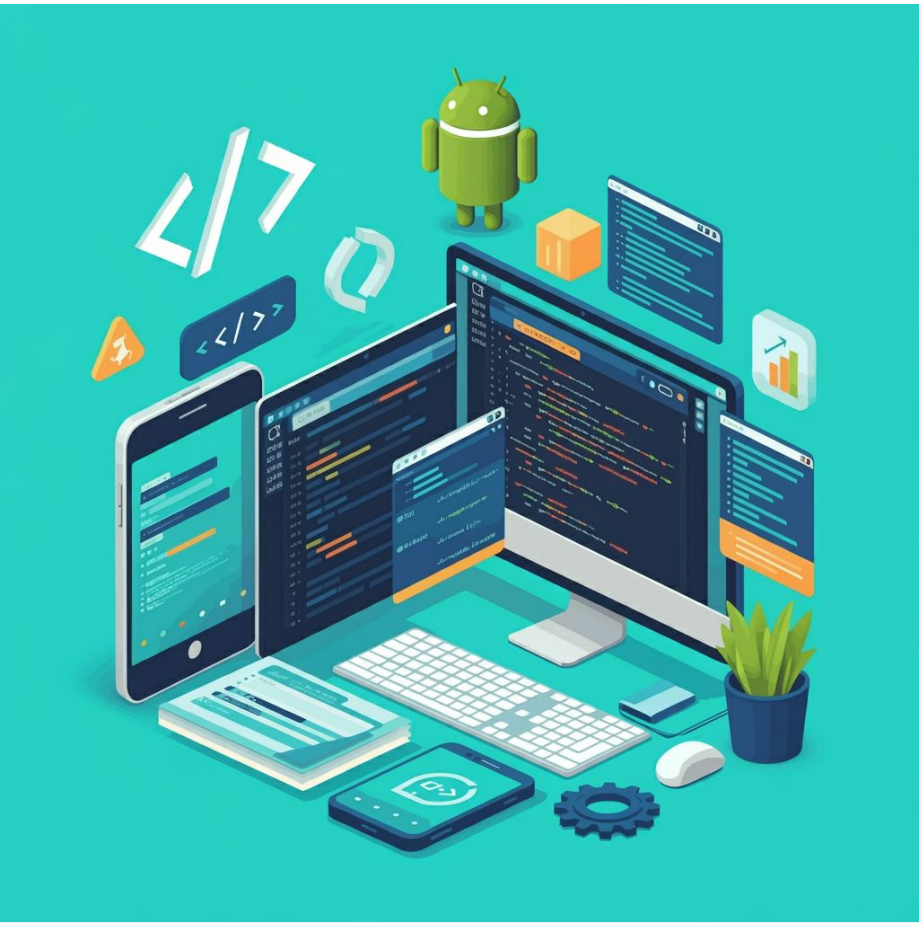
- GRIFFITHS D. Use a Cabeça! Desenvolvendo Para Android. 2 ed. Rio de Janeiro: Alta Books, 2019.

- GLAUBER, N. Dominando o Android com Kotlin. São Paulo: Novatec, 2019.

Avaliações

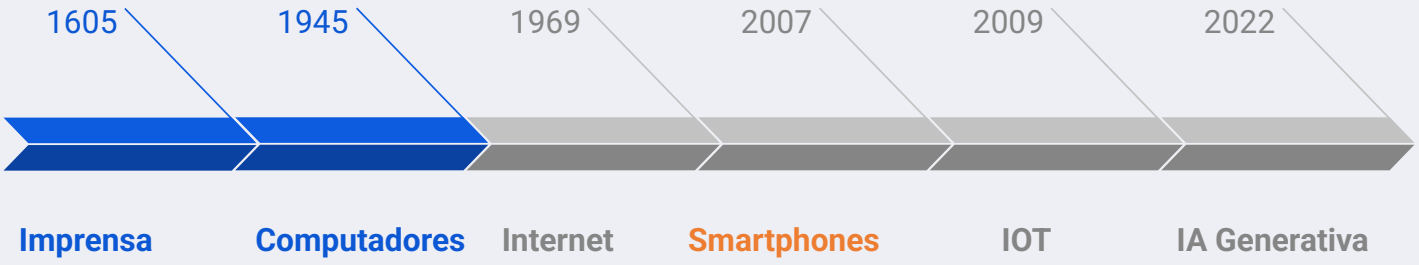
ATD 1 E 2

ATD3 - PROJETO



Fundamentos Essenciais

ANTES DO MOBILE



SURGIMENTO MÓVEL

- **Ipod e Iphone, Android e Preço**

  Iphone - 29 de junho de 2007

  Android - 22 de outubro de 2008

DISPOSITIVOS MÓVEIS E PESSOAS

- **Smartphones - RV/AR**
  - Uso do dia a dia.
  - Recursos
  - Agilidade
  - Jogos

OS DISPOSITIVOS MÓVEIS NA ATUALIDADE

Inovação Tecnológica

Dispositivos cada vez mais poderosos e versáteis.

Integração na Vida Diária

Uso constante para comunicação, entretenimento e trabalho.

Mudanças no Comportamento

Impacto na forma como interagimos com o mundo e nos relacionamos.

DISPOSITIVOS MÓVEIS E SEGURANÇA

- **Smartphones**
  - Hardware
  - S.O
  - Software
  - Conectividade Dados, Bluetooth e Wifi

COMPUTADORES E DESENVOLVEDORES

O desenvolvimento para dispositivos móveis, até a presente data, vem sendo focado nas plataformas iOS e Android.

Então em qual programar?

Qual o escopo do projeto?

O QUE O CLIENTE ESPERA?

O que vem ocorrendo no desenvolvimento para dispositivos móveis é que, ao desenvolver uma aplicação para um dispositivo e uma plataforma, esperamos que esta aplicação possa funcionar em qualquer dispositivo, de qualquer plataforma.

COMO AS LINGUAGENS FUNCIONAM

O objetivo é partir para uma ferramenta que possibilite o desenvolvimento seja cross-platform.

- Ambiente Web Híbrida -React Native e Ionic (Capacitor)
- Aplicativos Nativos - Swift, Kotlin e Java
- Multiplataforma - Dart Flutter e MAUI

QUAL ESCOLHER?

A escolha da linguagem e framework ideal depende das suas necessidades específicas. Considere os seguintes fatores:

- **Performance**: Se a performance é crítica, o desenvolvimento nativo é a melhor opção.
- **Tempo de desenvolvimento**: Se você precisa lançar um aplicativo rapidamente, o desenvolvimento híbrido

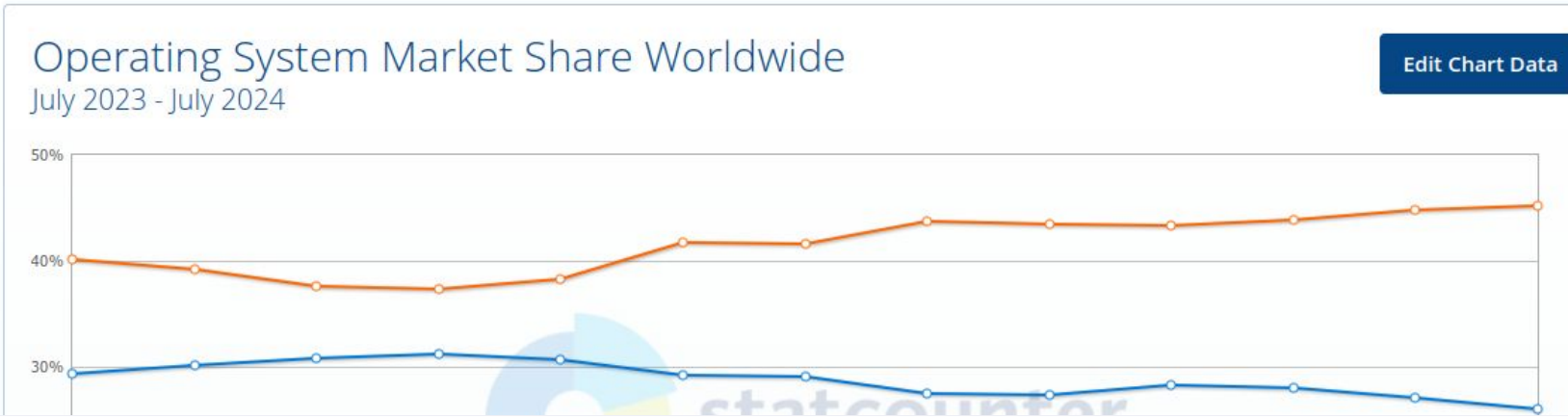
QUAL ESCOLHER?

- **Orçamento**: O desenvolvimento nativo geralmente requer mais recursos, enquanto o híbrido e o multiplataforma podem ser mais econômicos.

- **Experiência do usuário**: Aplicativos nativos oferecem a melhor experiência do usuário.

QUAL O SISTEMA OPERACIONAL MAIS UTILIZADO DO MUNDO?





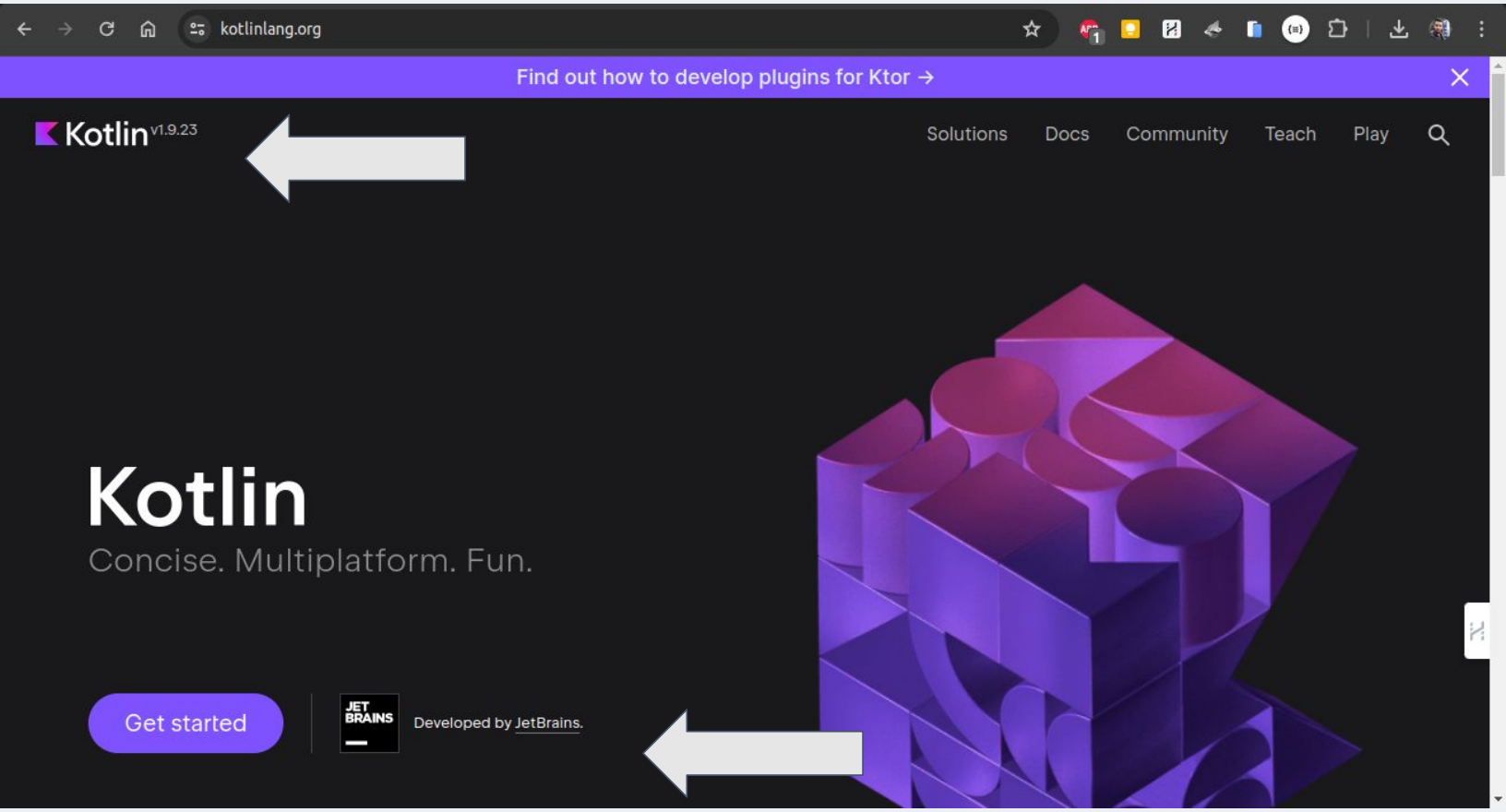
TP1 - KOTLIN

1. Crie um App No Formato de um Portfólio de Desenvolvedor.
   - Aba Inicial Com Sua Descrição (Sobre Você) LinkedIn
   - Abas Separadas Por Linguagens.
   - Utilize os Links dos Seus Projetos Versionados No Github



Introdução ao Kotlin

Kotlin é uma linguagem de programação moderna, concisa e expressiva que vem ganhando cada vez mais popularidade no desenvolvimento de aplicativos, especialmente para Android.



Valorização do Kotlin

O Kotlin começou a ganhar popularidade gradualmente após seu lançamento em **2011**, mas sua adoção decolou em **2017**, quando a **Google anunciou o suporte oficial ao Kotlin para o desenvolvimento de aplicativos Android**. Isso foi significativo, pois muitos desenvolvedores Android estavam procurando alternativas ao Java, que era a principal linguagem para desenvolvimento Android até então.

Apresentação da linguagem Kotlin e suas vantagens.

- Linguagem moderna e concisa para desenvolvimento Android.
- Combina características de linguagens funcionais e orientadas a objetos.
- Sintaxe simples e intuitiva, aumentando a produtividade.
- Foco em segurança, legibilidade e código limpo.

Características da linguagem

1. **Segurança de Nulabilidade**
2. **Sintaxe Concisa**
3. **Lambda expressions**

Características da linguagem

4 **Integração Perfeita Com Java**

5 **Migração Gradual**

6 **Compartilhamento Em Plataformas**

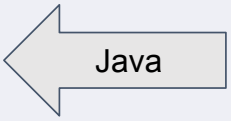
"Olá, Mundo!" em Kotlin

```
fun main() {
    println("Bem-vindo ao Kotlin!")
}
```

- fun main(): Ponto de entrada do programa.
- println(): Função para exibir mensagens no console.
- Demonstra a simplicidade da linguagem para tarefas básicas.

Interoperabilidade

```
public class Calculadora{
    public int somar(int a, int b) {
        return a + b;
    }
}
```




```
val calc = Calculadora()
val resultado = calc.somar(2, 2)
println(resultado)
```



Variáveis e Tipos de Dados

Declaração e tipos de dados em Kotlin.

- val: Declara uma variável imutável (valor constante).
- var: Declara uma variável mutável (valor pode ser alterado).
- Tipos de dados: Int, Double, String, Boolean, etc.
- Inferência de tipo: Kotlin pode inferir o tipo da variável automaticamente.

Exemplo:

```
val numeroConta: Int = 12345
var saldo: Double = 1000.0
```

Variáveis Val e Var - inferred type

```
var idade: Int = 22
idade = 18
```

```
val idade = 22
idade = 26
```

Kotlin

//Esse código não compila porque estou tentando alterar o valor de uma variável imutável

Casting

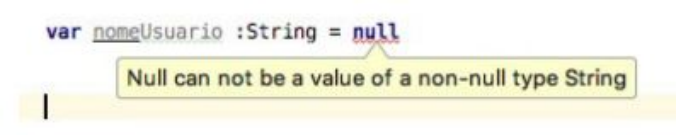
Kotlin

```
val x: Int = 10;
var y: Double = x.toDouble() //Retorna um objeto Double a partir do valor de x
var z: Float = x.toFloat() //Retorna um objeto Float a partir do valor de x
var a: String = x.toString() //Retorna um objeto String a partir do valor de x
```

Referências Nulas

```
String nomeUsuario = null;
```

Java




```
var nomeUsuario: String? = null
if (nomeUsuario!= null) {
    println(nomeUsuario.length)
}
```

Kotlin

Referências Nulas - Safe Call

```
var nomeUsuario: String? = null
println(nomeUsuario?.length)
```

```
var nomeUsuario: String? = null
println(nomeUsuario!!.length)
```

Kotlin

Estruturas de Controle

Controle de fluxo com a estrutura condicional when.

Além do if e else tradicional o kotlin conta com o when.

Definição:

when: Estrutura poderosa para múltiplas condições.

Mais concisa e legível que if-else if-else.

Permite avaliar diferentes valores de uma expressão.



Estruturas de Controle

Controle de fluxo com a estrutura condicional if.

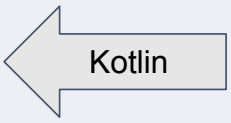
```
val senha = "123"
if (senha == "123") {
    println("Acesso concedido")
} else {
    println("Senha incorreta")
}
```

Kotlin

Estruturas de Controle

Controle de fluxo com a estrutura condicional when.

```
when    (x)   {
      1     ->    print("x   ==   1")
      2     ->    print("x   ==   2")
else  ->    {
      print("x   possui    outro     valor")
}
}
```



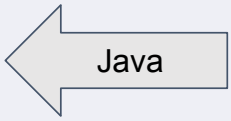
Estruturas de Controle

Controle de fluxo com a estrutura condicional when.

```
fun escolherPagamento(metodo: String) {
    when (metodo) {
        "cartao" -> println("Cartão selecionado.")
        "boleto" -> println("Boleto selecionado.")
        else -> println("Método inválido.")
    }
}
```

Uma Linguagem Concisa

```
Button buttonlogin = (Button) findViewById(R.id.button_login);
buttonlogin.setOnClickListener(new View.OnClickListener() {
@Override
public void onClick(View view) {
    efetuarLogin();
}
});
```



Uma Linguagem Concisa

```
val buttonlogin = findViewById<Button>(R.id.button_login);
buttonlogin.setOnClickListener {
    efetuarLogin()
}
```

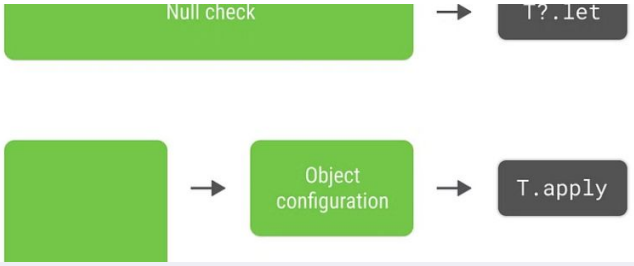


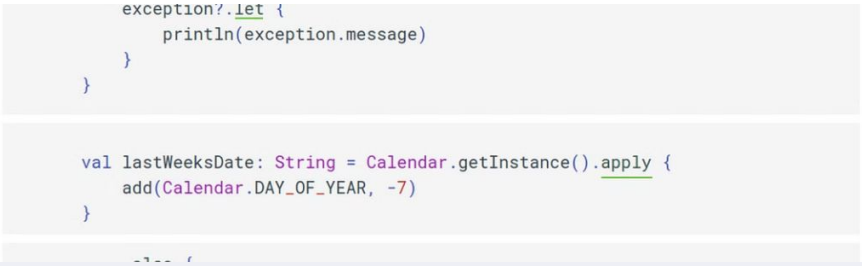
Estrutura de Repetição

```
val   lista  =     listOf(1,2,3,4)
for   (i     in    lista){
      println("Item:  $i")
}
```




```
var   x     =     0
while       (x    <     10)   {
      println(x.toString())
      x++
}
```





```
exception?.let {
    println(exception.message)
}
}
```

```
val lastWeeksDate: String = Calendar.getInstance().apply {
    add(Calendar.DAY_OF_YEAR, -7)
}
```

Funções e Lambda Expressions

As lambda expressions em Kotlin são poderosas ferramentas que permitem escrever código funcional de forma concisa e legível.

Exemplo:

```
val listaCompras = listOf("Leite", "Pão", "Queijo")
listaCompras.forEach { item -> println("Comprar: $item") }
```

Funções

```
fun somar(n1:Int , n2: Int): Int {
    return n1 + n2
}
//E para chamar a função:
val resultado= somar(5 , 7)

//Retorno Vazio:
fun imprimir(texto: String): Unit {
    println(texto)
}
```



POO em Kotlin: Classes e Objetos

Programação Orientada a Objetos com Kotlin.

Definição:

- Classe: Modelo para criar objetos (instâncias da classe).
- Objeto: Possui propriedades (dados) e comportamentos (funções).
- Encapsulamento: Protege os dados e expõe comportamentos controlados.

POO- Kotlin

```
class Jogo {
    var titulo = ""
    var capa = ""
    val descricao = ""

    override fun toString(): String {
        return "Meu Jogo: \n" +
                "Título: $titulo \n" +
                "Capa: $capa \n" +
                "Descricao: $descricao"
    }
}
```



POO- Kotlin

```
class Jogo(val titulo:String, val capa:String) {
    val descricao = ""

    override fun toString(): String {
        return "Meu Jogo: \n" +
                "Título: $titulo \n" +
                "Capa: $capa \n" +
                "Descricao: $descricao"
    }
}
```



POO- Kotlin

```
fun main() {

// código omitido...

val meuJogo = Jogo(
        "Batman: Arkham Asylum Game of the Year Edition",
        "https:\\/\\/cdn.cloud...")
    println(meuJogo)
}
```

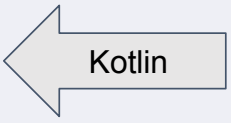


POO- Kotlin

Exemplo:

```
data class Tarefa(val titulo: String, var status: String)

data class Cliente(var email: String, var nomeUsuario: String)
```

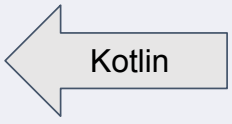


Aplicações e Ecossistema Kotlin

Android	Kotlin é a linguagem oficial para desenvolvimento de aplicativos Android, oferecendo melhor produtividade e segurança.
Backend	Kotlin pode ser usado para desenvolvimento de aplicações backend, integrando-se bem com frameworks como Spring.
Multiplataforma	Com Kotlin Multiplatform, é possível criar aplicações que rodam em diferentes plataformas, como Android, iOS e web.
Ecossistema	O ecossistema Kotlin é amplo e em constante evolução, com ferramentas, bibliotecas e recursos de desenvolvimento.

Kotlin Dependências

```
<dependency>
 <groupId>com.google.code.gson</groupId>
 <artifactId>gson</artifactId>
 <version>2.10.1</version>
 <scope>compile</scope>
</dependency>
```



Kotlin Dependências

```
import com.google.gson.annotations.SerializedName
class Jogo(@SerelializedName("title") val titulo:String,
           @SerelializedName("thumb") val capa:String) {
    val descricao = ""
    override fun toString(): String {
        return "Meu Jogo: \n" +
                "Título: $titulo \n" +
                "Capa: $capa \n" +
                "Descricao: $descricao"
    }
}
```



Kotlin Dependências

```
import com.google.gson.Gson

val json = response.body()
    println(json)
    val gson = Gson()
    val meuJogo = gson.fromJson(json, Jogo::class.java)
    println(meuJogo)
```





Veja na Prática Uma Aplicação!